Line-by-Line Mathematical Guide for HAETAE_Scheme.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

HAETAE instantiation of the generic signature interface.

How To Read This Script

Read this file as an adapter. It aligns HAETAE key generation, signing, and verification with the generic games without doing the security proof locally.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	`require import AllCore.`	Imports AllCore. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	`require import Sig_ROM HAETAE_Params HAETAE_Algebra HAETAE_Distributions.`	Imports Sig_ROM HAETAE_Params HAETAE_Algebra HAETAE_Distributions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	`require import HAETAE_ROM.`	Imports HAETAE_ROM. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
4	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5	`theory HAETAE_Scheme.`	Begins the namespace HAETAE_Scheme, grouping the declarations as one checked theory.
6	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7	`import HAETAE_Params.`	Opens HAETAE_Params so its names can be used without qualification in the following proof.
8	`import HAETAE_Algebra.`	Opens HAETAE_Algebra so its names can be used without qualification in the following proof.
9	`import HAETAE_Distributions.`	Opens HAETAE_Distributions so its names can be used without qualification in the following proof.
10	`import HAETAE_ROM.`	Opens HAETAE_ROM so its names can be used without qualification in the following proof.
11	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
12	`clone import Sig_ROM as SIG with`	Clones and opens a parameterized EasyCrypt theory, producing a specialized instance used below.
13	`type pkey <- pkey,`	Declares carrier set pkey. EasyCrypt will type-check all later functions and games over this mathematical domain.
14	`type skey <- skey,`	Declares carrier set skey. EasyCrypt will type-check all later functions and games over this mathematical domain.

Line	EasyCrypt source	Mathematical reading
15	`type message <- message,`	Declares carrier set message. EasyCrypt will type-check all later functions and games over this mathematical domain.
16	`type context <- context,`	Declares carrier set context. EasyCrypt will type-check all later functions and games over this mathematical domain.
17	`type signature <- signature,`	Declares carrier set signature. EasyCrypt will type-check all later functions and games over this mathematical domain.
18	`type ro_query <- ro_query,`	Declares carrier set ro_query. EasyCrypt will type-check all later functions and games over this mathematical domain.
19	`type ro_output <- ro_output,`	Declares carrier set ro_output. EasyCrypt will type-check all later functions and games over this mathematical domain.
20	`op dro_output <- dro_output.`	Defines operation dro_output, a total mathematical function or abbreviation used by later declarations.
21	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
22	`op haetae_mode :  mode.`	Defines operation haetae_mode, a total mathematical function or abbreviation used by later declarations.
23	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
24	`module HAETAE(H : SIG.POracle) = {`	Defines module HAETAE, a probabilistic program or game used to create events and success probabilities.
25	`proc kg() :  pkey * skey = {`	Defines procedure kg. Its body is a probabilistic program whose result becomes part of a game event.
26	`var sd :  seed;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
27	`var rhoprime :  seed;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
28	`var kp :  pkey * skey;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
29	`var ro_y :  ro_output;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
30	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
31	`sd <$ dseed;`	Samples from a distribution. Mathematically this introduces a random variable with the named law.
32	`rhoprime <- haetae_keygen_rhoprime sd;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
33	`ro_y <@ H.get(matrix_expand_query haetae_mode rhoprime);`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
34	`kp <- keygen_internal haetae_mode rhoprime;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
35	`return kp;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
36	`}`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
37	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
38	`proc sign(sk :  skey, m :  message, ctx :  context) :  signature = {`	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
39	`var coins :  random_coins;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
40	`var pk :  pkey;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
41	`var highbits :  polyveck;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
42	`var lowbits :  poly;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
43	`var mu :  crh;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
44	`var ro_y :  ro_output;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
45	`var sig :  signature;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
46	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
47	`coins <$ drandom_coins;`	Samples from a distribution. Mathematically this introduces a random variable with the named law.

Line	EasyCrypt source	Mathematical reading
48	`ro_y <@ H.get(sampler_expand_query coins);`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
49	`coins <- ro_signing_coins ro_y;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
50	`pk <- public_key_of_secret haetae_mode sk;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
51	`ro_y <@ H.get(message_hash_query pk ctx m);`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
52	`mu <- ro_message_hash ro_y;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
53	`highbits <- commitment_highbits haetae_mode sk m ctx coins;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
54	`lowbits <- commitment_lowbits haetae_mode sk m ctx coins;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
55	`ro_y <@ H.get(challenge_hash_query haetae_mode highbits lowbits mu);`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
56	`sig <- sign_internal haetae_mode sk m ctx coins;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
57	`return sig;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
58	`}`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
59	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
60	`proc verify(pk :  pkey, m :  message, ctx :  context,`	Defines procedure verify. Its body is a probabilistic program whose result becomes part of a game event.
61	`sig :  signature) :  bool = {`	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
62	`return verify_internal haetae_mode pk m ctx sig;`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
63	`}`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
64	`}.`	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
65	*blank*	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
66	`end HAETAE_Scheme.`	Closes the current theory or module block.